ABACUS PRIVATE LIMITED

Department of Computer Science

PROJECT REPORT

Fake News Detection Using Python & Machine Learning

4th Semester, 2nd Year

Under the Guidance of:

Subratnath Sharma

Submitted By:

Prakash Chandra Barick

Mihir Das

Biswajit Pradhan

Aditya Kumar Behera

Dibya Santoshmay Das

Ganesh Kumar Maghi

Certificate

This is to certify that the project report entitled **"Fake News Detection Using Python & Machine Learning"** submitted by Prakash Chandra Barick, Mihir Das, Biswajit Pradhan, Aditya Kumar Behera, Dibya Santoshmay Das, and Ganesh Kumar Maghi in partial fulfillment of the requirements for the 4th Semester, 2nd Year of the Bachelor of Science (BSc) degree in Computer Science, is an authentic work carried out by them under my supervision and guidance.

To the best of my knowledge, the matter embodied in this project report has not been submitted to any other University or Institute for the award of any degree or diploma.

Subratnath Sharma
Project Guide
Department of Computer Science
Abacus Private Limited

Declaration

We hereby declare that the project entitled **"Fake News Detection Using Python & Machine Learning"** submitted to the Department of Computer Science is a record of an original work done by us under the guidance of Subratnath Sharma. This project is submitted in the partial fulfillment of the requirements for the 4th Semester, 2nd Year of our undergraduate degree.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Date: _____

Place: Baripada, Odisha

Signatures:

1. Prakash Chandra Barick
2. Mihir Das
3. Biswajit Pradhan
4. Aditya Kumar Behera
5. Dibya Santoshmay Das
6. Ganesh Kumar Maghi

Acknowledgement

We would like to express our profound gratitude and deep regards to our guide **Subratnath Sharma** for his exemplary guidance, monitoring, and constant encouragement throughout the course of this project. The blessing, help, and guidance given by him time to time shall carry us a long way in the journey of life on which we are about to embark.

We also take this opportunity to express a deep sense of gratitude to the Department of Computer Science and Abacus Private Limited for providing excellent infrastructure, computer laboratories, and necessary facilities that helped us in completing this project successfully.

Finally, we thank our parents, friends, and peers for their constant support, motivation, and valuable suggestions during the development of this text classification model.

Abstract

The rapid expansion of internet connectivity and the widespread use of social media platforms have fundamentally altered the way news and information are disseminated. While this democratization of information has many benefits, it has also facilitated the unchecked spread of "fake news"—deliberately misleading or fabricated information presented as legitimate reporting. The proliferation of fake news poses a significant threat to public discourse and critical decision-making.

This project addresses this issue by developing an automated Fake News Detection system using Machine Learning techniques. Specifically, we utilize Python to build a robust binary classification pipeline. The core of our methodology involves Natural Language Processing (NLP) to extract meaningful features from raw text data. We employ TF-IDF (Term Frequency-Inverse Document Frequency) vectorization to convert textual news articles into numerical representations. Subsequently, a Logistic Regression model is trained on a labeled dataset consisting of thousands of real and fake news articles.

The workflow encompasses comprehensive data preprocessing, merging of datasets, shuffling, and rigorous evaluation using an 80-20 train-test split methodology. The implemented model demonstrates high efficiency and accuracy in classifying unseen news articles, highlighting the practical applicability of Artificial Intelligence in mitigating the spread of misinformation.

1. Introduction

1.1 Background

In recent years, the consumption of news has heavily migrated from traditional print and broadcast media to online platforms. Social media networks, blogs, and independent news websites allow news to spread at an unprecedented velocity. However, this lack of rigorous editorial oversight allows malicious actors to easily publish and amplify false narratives. Fake news can be categorized into various types, including clickbait, propaganda, satire, and completely fabricated stories designed to manipulate public opinion or generate ad revenue.

1.2 Problem Statement

The manual verification of news articles is a highly labor-intensive and time-consuming process. Human fact-checking organizations cannot scale their operations to match the vast volume of content generated daily on platforms like Twitter, Facebook, and WhatsApp. Therefore, the core problem lies in the absence of an automated, scalable, and highly accurate computational system capable of evaluating the authenticity of a news article in real-time based purely on its textual content.

1.3 Objectives

The primary objectives of this project are as follows:

- To collect and preprocess a substantial dataset of labeled real and fake news articles to ensure balanced training data.
- To implement Natural Language Processing (NLP) techniques, specifically TF-IDF vectorization, for effective textual feature extraction.
- To train a Logistic Regression machine learning model optimized for binary text classification.
- To evaluate the performance of the model using standard metrics such as accuracy, precision, recall, and f1-score.

- To develop a clean, understandable, and modular Python codebase that demonstrates real-world software development practices.

1.4 Need of Project

Automated fake news detection is crucial for preserving the integrity of information online. By providing users with a computational tool to quickly assess the credibility of an article, we can significantly reduce the viral spread of misinformation. This project serves as a foundational step toward building more complex, real-time filtering systems that could eventually be integrated into social media feed algorithms or browser extensions, empowering consumers to make informed reading decisions.

2. Literature Review

The challenge of detecting fake news has garnered significant attention from both the academic and tech communities over the last decade. Early approaches heavily relied on manual fact-checking platforms and crowdsourced verification. However, with the advent of Machine Learning (ML) and advancements in Natural Language Processing (NLP), automated text classification rapidly became the industry standard.

Several studies have explored different algorithms for this specific classification task. Naïve Bayes classifiers have been widely used due to their simplicity, fast training times, and effectiveness in text categorization, fundamentally relying on the assumption of feature independence. Support Vector Machines (SVM) have also shown strong performance in high-dimensional vector spaces, making them highly suitable for handling large TF-IDF representations. More recently, deep learning architectures, such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, have achieved state-of-the-art results by capturing the sequential context and syntactic structure of words within a document.

Despite the success of deep neural networks, they require immense computational resources, GPUs, and extremely large datasets to train effectively without overfitting. For an undergraduate project, selecting an algorithm that perfectly balances predictive performance, model interpretability, and computational efficiency is vital. Logistic Regression, when paired with robust TF-IDF vectorization, has been proven in academic literature to provide an exceptionally strong baseline for binary text classification. It often achieves accuracy rates rivaling more complex models while requiring only a fraction of the training time and CPU resources.

3. System Analysis

3.1 Technology Used

The fake news detection system is developed utilizing the following core data science and software technologies:

- **Python 3.x:** The primary programming language chosen due to its intuitive syntax and rich ecosystem of data science and machine learning libraries.
- **Pandas:** A powerful library utilized extensively for loading CSV datasets, data manipulation, merging frames, and handling missing values.
- **NumPy:** Used behind the scenes for highly efficient numerical operations and multi-dimensional array manipulations.
- **Scikit-Learn (sklearn):** The central machine learning library employed for generating the TF-IDF vectorization, training the Logistic Regression classifier, and executing performance evaluation metrics.
- **Regular Expressions (re):** A built-in Python module utilized for advanced string matching and text preprocessing, including the removal of special characters, URLs, and punctuation.

3.2 System Requirements

To execute the text processing and model training scripts efficiently, the following system specifications are recommended:

- **Hardware Requirements:** A modern multi-core processor (e.g., AMD Ryzen 3 or higher), a minimum of 8GB of RAM (essential to handle large sparse matrices generated by text vectorization in memory), and at least 2GB of free storage space for datasets.
- **Software Requirements:** An Operating System supporting Python (Windows 10/11, Linux, or macOS), Python version 3.8 or higher, and an Integrated Development Environment (IDE) such as Jupyter Notebook, VS Code, or PyCharm.

4. System Design & Methodology

4.1 Dataset Description

The project utilizes a comprehensive dataset consisting of two distinct CSV files: one containing thousands of confirmed fake news articles and another containing real, verified news articles from reputable publishers. Each dataset includes essential features such as the `title` of the article, the `author`, the main body `text`, and the target `label`. Ensuring a balanced dataset (an equal proportion of real and fake articles) is critical to prevent the Logistic Regression model from developing a statistical bias toward the majority class.

4.2 Data Preprocessing

Raw text data scraped from the web is highly unstructured, noisy, and cannot be fed directly into a mathematical machine learning model. The preprocessing pipeline is a crucial step and includes:

- **Merging and Labeling:** Combining the fake news dataframe and the real news dataframe into a single unified dataset, assigning binary labels (0 for Fake, 1 for Real).
- **Handling Missing Values:** Systematically dropping rows with null values or filling them with empty strings to prevent execution errors during vectorization.
- **Text Cleaning:** Converting all string characters to lowercase to maintain uniformity. We then use regular expressions to strip out punctuation marks, numerical digits, HTML tags, and hyperlink URLs.
- **Shuffling:** Randomizing the order of the dataset to ensure that the model does not learn any sequential bias based on how the files were appended.

4.3 General Methodology

The overall workflow strictly follows a standard machine learning pipeline architecture. First, the raw textual data is collected and preprocessed as previously described. Next, the cleaned text is transformed into a numerical matrix format using TF-IDF. The dataset is then divided into an independent training set and a testing set. The training set is fed into the Logistic Regression algorithm to optimize its internal weights. Finally, the model's predictive capability is rigorously evaluated on the unseen testing set to verify its real-world generalization.

5. Implementation Details

5.1 TF-IDF Vectorization

To apply any machine learning algorithm, raw text must first be converted into numerical vectors. Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical measure that evaluates how relevant a specific word is to a document within a larger collection of documents (the corpus).

Term Frequency (TF): Measures how frequently a term appears in a given document. It is calculated as:

$$TF(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total number of words in document } d)$$

Inverse Document Frequency (IDF): Measures how important a term is across the entire corpus. It weighs down highly frequent, uninformative words (like "the", "is") and scales up rare, descriptive words.

$$IDF(t, D) = \log_e(\text{Total number of documents} / \text{Number of documents with term } t \text{ in it})$$

By multiplying these two metrics ($TF \times IDF$), we obtain a feature matrix where rare, highly descriptive words receive significantly higher mathematical weights, making it easier for the model to distinguish between text categories.

5.2 Logistic Regression Model

Logistic Regression is a fundamental supervised learning classification algorithm used to predict the probability of a categorical dependent variable. For our binary classification problem (Real vs. Fake), it utilizes the sigmoid function to map predicted linear values to a strict probability range between 0 and 1.

The hypothesis function is mathematically defined as:

$$h(x) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)})$$

Where the weights (β) are iteratively learned and optimized during the training phase using a loss optimization algorithm, such as Gradient Descent, to minimize the log-loss error.

5.3 Train-Test Split

To properly evaluate the model's ability to generalize to new, unseen news articles, the dataset is divided using the `train_test_split` function from the Scikit-Learn library. A standard ratio of 80% for training data and 20% for testing data is employed. This ensures the model has enough data to learn the complex patterns of fake news while retaining a sufficiently large, untouched dataset to validate its accuracy impartially.

6. Results and Discussion

6.1 Results and Accuracy

Upon training the Logistic Regression model and evaluating its performance on the 20% testing set, the system achieved an exceptionally high degree of accuracy. The model performance heavily depends on the balanced nature of the initial datasets, the strict quality of the regex preprocessing pipeline, and the mathematical effectiveness of the TF-IDF feature extraction.

Accuracy is formally measured using the `accuracy_score` metric from sklearn, with the system routinely achieving upwards of 90-95% accuracy on standard benchmark datasets. The classification report further indicated high Precision and Recall scores, demonstrating that the chosen NLP approach is highly effective for this specific problem domain and minimizes false positives.

6.2 Advantages

- **High Computational Efficiency:** Logistic Regression trains rapidly (often in seconds) and requires minimal CPU and RAM overhead compared to deep neural networks.
- **Model Interpretability:** Unlike "black box" deep learning models, the weights assigned to individual TF-IDF features in Logistic Regression allow developers to directly analyze which specific words are most indicative of fake news.
- **Ease of Deployment:** The lightweight nature of the trained model file means it can easily be deployed as a web API or integrated into lightweight client-side applications.

6.3 Limitations

- **Lack of Contextual Understanding:** TF-IDF fundamentally treats text as a "bag of words." It completely loses the sequential context, grammar, and semantic meaning of complex sentences.
- **Vulnerability to Sarcasm and Parody:** The model struggles with highly nuanced text, such as satire or sarcastic articles, as it relies purely on word frequencies rather than deep linguistic comprehension.

6.4 Applications and Future Scope

The current Python-based system can be effectively utilized by news aggregator platforms to automatically flag potentially suspicious articles for human editorial review. In the future, the scope of this project can be significantly expanded by integrating more advanced deep learning models such as BERT (Bidirectional Encoder Representations from Transformers) to capture deep semantic meaning. Additionally, deploying the model as a real-time browser extension that automatically scrapes and analyzes web page text on the fly would vastly enhance its practical consumer utility.

7. Conclusion

This academic project successfully demonstrates the practical application of Python and Machine Learning techniques to the critical, real-world problem of Fake News Detection. By systematically preprocessing noisy textual data and applying TF-IDF vectorization, we were able to transform qualitative, unstructured news articles into strictly quantitative numerical features.

The implementation of the Logistic Regression model proved to be a highly efficient, interpretable, and accurate classifier, effectively distinguishing between genuine reporting and fabricated stories. Through this endeavor, the theoretical concepts of Natural Language Processing, data cleaning, and binary classification have been translated into a functional, modular software pipeline. While certain limitations exist regarding the model's contextual understanding of human language, this project establishes a robust computational foundation for building more sophisticated AI-driven verification systems in the future, ultimately contributing to the reduction of misinformation in the digital age.

References

1. Conroy, N. J., Rubin, V. L., & Chen, Y. (2015). "Automatic deception detection: Methods for finding fake news." *Proceedings of the Association for Information Science and Technology*, 52(1), 1-4.
2. Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). "Fake news detection on social media: A data mining perspective." *ACM SIGKDD Explorations Newsletter*, 19(1), 22-36.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825-2830.
4. McKinney, W. (2010). "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, 51-56.
5. Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing* (3rd ed. draft). Pearson.

Appendix: Full Python Code

The complete Python source code utilized for dataset handling, regex preprocessing, TF-IDF vectorization, model training, and performance evaluation is provided below for academic reference.

```
import pandas as pd
import numpy as np
import re
import string
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# 1. Load the Datasets
try:
    fake_data = pd.read_csv('Fake.csv')
    real_data = pd.read_csv('True.csv')
except FileNotFoundError:
    print("Error: Dataset files not found in the current directory.")
    exit()

# 2. Assign Binary Labels (0 = Fake, 1 = Real)
fake_data['class'] = 0
real_data['class'] = 1

# Optional: Drop unnecessary columns to save memory
fake_data = fake_data.drop(['title', 'subject', 'date'], axis=1)
real_data = real_data.drop(['title', 'subject', 'date'], axis=1)

# 3. Merge and Shuffle the Data
news_dataset = pd.concat([fake_data, real_data], axis=0)
# Shuffle the dataset to prevent sequential bias
news_dataset = news_dataset.sample(frac=1).reset_index(drop=True)

# 4. Text Preprocessing Function
def wordopt(text):
    text = text.lower() # Convert to lowercase
    text = re.sub('\[.*?\]', '', text) # Remove text in square brackets
    text = re.sub("\W", " ", text) # Remove special characters
    text = re.sub('https?://\S+|www\.\S+', '', text) # Remove URLs
    text = re.sub('<.*?>+', '', text) # Remove HTML tags
    text = re.sub('[%s]' % re.escape(string.punctuation), '', text) # Remove
punctuation
    text = re.sub('
', '', text) # Remove newlines
    text = re.sub('\w*\d\w*', '', text) # Remove words containing numbers
```

```
    return text

# Apply the preprocessing function to the text column
print("Preprocessing text data...")
news_dataset['text'] = news_dataset['text'].apply(wordopt)

# 5. Define Independent (X) and Dependent (Y) Variables
X = news_dataset['text']
Y = news_dataset['class']

# 6. Perform Train-Test Split (80% Train, 20% Test)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
random_state=42)

# 7. TF-IDF Vectorization Feature Extraction
vectorization = TfidfVectorizer()
xv_train = vectorization.fit_transform(X_train)
xv_test = vectorization.transform(X_test)

# 8. Model Initialization and Training
LR = LogisticRegression()
print("Training the Logistic Regression model...")
LR.fit(xv_train, Y_train)

# 9. Prediction and Evaluation
pred_lr = LR.predict(xv_test)
accuracy = accuracy_score(Y_test, pred_lr)

print("
--- Model Evaluation Results ---")
print(f"Accuracy Score: {accuracy * 100:.2f}%")
print("
Classification Report:")
print(classification_report(Y_test, pred_lr))
```